JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.reflect

Class AccessibleObject

java.lang.Object
    java.lang.reflect.AccessibleObject

All Implemented Interfaces:

AnnotatedElement

Direct Known Subclasses:

Executable, Field

```
public class AccessibleObject
extends Object
implements AnnotatedElement
```

The AccessibleObject class is the base class for Field, Method and Constructor objects. It provides the ability to flag a reflected object as suppressing default Java language access control checks when it is used. The access checks--for public, default (package) access, protected, and private members--are performed when Fields, Methods or Constructors are used to set or get fields, to invoke methods, or to create and initialize new instances of classes, respectively.

Setting the `accessible` flag in a reflected object permits sophisticated applications with sufficient privilege, such as Java Object Serialization or other persistence mechanisms,
to manipulate objects in a manner that would normally be prohibited.

By default, a reflected object is *not* accessible.

Since:

1.2

See Also:

Field, Method, Constructor, ReflectPermission

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AccessibleObject()`	Constructor: only used by the Java Virtual Machine.

Method Summary

Concrete Methods

Modifier and Type	Method	De
`<T extends Annotation> T`	`getAnnotation(Class<T> annotationClass)`	R el ar fo sp ty ar ar is el
`Annotation[]`	`getAnnotations()`	R ar th *p* th
`<T extends Annotation> T[]`	`getAnnotationsByType(Class<T> annotationClass)`	R ar th *as* w el
`<T extends Annotation> T`	`getDeclaredAnnotation(Class<T> annotationClass)`	R el ar fo sp ty ar ar is

		p n
Annotation[]	**getDeclaredAnnotations**()	R a th *d* *p* th
<T extends **Annotation**> T[]	**getDeclaredAnnotationsByType**(**Class**<T> annotationClass)	R el a fo sp ty a a *d* *p* *in* *p*
boolean	**isAccessible**()	G va a fl o
boolean	**isAnnotationPresent**(**Class**<? extends **Annotation**> annotationClass)	R if a fo sp ty *p* th el
static void	**setAccessible**(**AccessibleObject**[] array, boolean flag)	C m se a fl a o a se

void	**setAccessible**(boolean flag)	S a fl o in b v

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AccessibleObject

```
protected AccessibleObject()
```

Constructor: only used by the Java Virtual Machine.

Method Detail

setAccessible

```
public static void setAccessible(AccessibleObject[] array,
                                 boolean flag)
                          throws SecurityException
```

Convenience method to set the `accessible` flag for an array of objects with a single security check (for efficiency).

First, if there is a security manager, its `checkPermission` method is called with a `ReflectPermission("suppressAccessChecks")` permission.

A `SecurityException` is raised if `flag` is `true` but accessibility of any of the elements of the input `array` may not be changed (for example, if the element object is a `Constructor` object for the class `Class`). In the event of such a SecurityException, the accessibility of objects is set to `flag` for array elements upto (and excluding) the element for which the exception occurred;

the accessibility of elements beyond (and including) the element for which the exception occurred is unchanged.

Parameters:

`array - the array of AccessibleObjects`

`flag - the new value for the accessible flag in each object`

Throws:

`SecurityException - if the request is denied.`

See Also:

`SecurityManager.checkPermission(java.security.Permission)`, `RuntimePermission`

setAccessible

```
public void setAccessible(boolean flag)
                   throws SecurityException
```

Set the `accessible` flag for this object to the indicated boolean value. A value of `true` indicates that the reflected object should suppress Java language access checking when it is used. A value of `false` indicates that the reflected object should enforce Java language access checks.

First, if there is a security manager, its `checkPermission` method is called with a `ReflectPermission("suppressAccessChecks")` permission.

A `SecurityException` is raised if `flag` is `true` but accessibility of this object may not be changed (for example, if this element object is a `Constructor` object for the class `Class`).

A `SecurityException` is raised if this object is a `Constructor` object for the class `java.lang.Class`, and `flag` is true.

Parameters:

`flag - the new value for the accessible flag`

Throws:

`SecurityException - if the request is denied.`

See Also:

`SecurityManager.checkPermission(java.security.Permission)`, `RuntimePermission`

isAccessible

```
public boolean isAccessible()
```

Get the value of the `accessible` flag for this object.

Returns:

the value of the object's accessible flag

getAnnotation

```
public <T extends Annotation> T getAnnotation(Class<T> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Specified by:

getAnnotation in interface AnnotatedElement

Type Parameters:

T - the type of the annotation to query for and return if present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

NullPointerException - if the given annotation class is null

Since:

1.5

isAnnotationPresent

```
public boolean isAnnotationPresent(Class<? extends Annotation> annotationClass)
```

Returns true if an annotation for the specified type is *present* on this element, else false. This method is designed primarily for convenient access to marker annotations.

The truth value returned by this method is equivalent to: `getAnnotation(annotationClass) != null`

The body of the default method is specified to be the code above.

Specified by:

isAnnotationPresent in interface AnnotatedElement

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

`true if an annotation for the specified annotation type is present on this element, else false`

Throws:

`NullPointerException - if the given annotation class is null`

Since:

1.5

getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType(Class<T> annotationClass)
```

Description copied from interface: `AnnotatedElement`

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotationsByType in interface AnnotatedElement`

Type Parameters:

`T - the type of the annotation to query for and return if present`

Parameters:

`annotationClass - the Class object corresponding to the annotation type`

Returns:

`all this element's annotations for the specified annotation type if associated with this element, else an array of length zero`

Throws:

`NullPointerException - if the given annotation class is null`

Since:

1.8

getAnnotations

```
public Annotation[] getAnnotations()
```

Description copied from interface: `AnnotatedElement`

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotations` in interface `AnnotatedElement`

Returns:

annotations present on this element

Since:

1.5

getDeclaredAnnotation

```
public <T extends Annotation> T getDeclaredAnnotation(Class<T> annotationClass)
```

Description copied from interface: `AnnotatedElement`

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

Specified by:

`getDeclaredAnnotation` in interface `AnnotatedElement`

Type Parameters:

T - the type of the annotation to query for and return if directly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if directly present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.8

getDeclaredAnnotationsByType

```
public <T extends Annotation> T[] getDeclaredAnnotationsByType(Class<T> annotatio
```

Description copied from interface: `AnnotatedElement`

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getDeclaredAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getDeclaredAnnotationsByType` in interface `AnnotatedElement`

Type Parameters:

`T - the type of the annotation to query for and return if directly or indirectly present`

Parameters:

`annotationClass - the Class object corresponding to the annotation type`

Returns:

`all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero`

Throws:

`NullPointerException - if the given annotation class is null`

Since:

`1.8`

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Description copied from interface: `AnnotatedElement`

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getDeclaredAnnotations` in interface `AnnotatedElement`

Returns:

`annotations directly present on this element`

Since:

1.5

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.